Server-Sent Events

Real-time, server-to-client streaming over plain HTTP

1. What is SSE?

SSE = Server-Sent Events — a web standard where the server **pushes data to the client over a single, long-lived HTTP connection**. The client just listens.

Think of it as:

- **Long polling** = client asks, waits for one reply, asks again. Many short conversations.
- **SSE** = client opens a phone line. Server keeps talking on it. One long conversation.

```
Long polling: request -> reply -> request -> reply -> request -> reply
SSE: request -> reply, reply, reply, reply, reply, ...
(all on the same connection)
```

> **■ Why SSE exists**
> Before SSE / WebSockets, real-time updates required hacks — short polling (wasteful), long polling (complex). SSE was added to HTML5 specifically for **one-way server -> client streaming**. Simpler than WebSockets when you only need pushes.

2. Common use cases

- Live notifications
- Stock tickers
- Live sports scores
- Activity feeds
- Build / deployment status streams
- **AI chat streaming** (ChatGPT, Claude — token-by-token streaming you see is SSE under the hood)

3. How SSE works under the hood

The server sends a response with a special MIME type and **never closes it**. Data flows in a tiny text protocol where each event looks like this:

```
data: hello

data: world
data: still streaming
```

Each event ends with a **blank line** (i.e. `\n\n`). The browser parses these as they arrive and fires events on your `EventSource` object.

Required HTTP response headers:

```
Content-Type: text/event-stream
Cache-Control: no-cache
Connection: keep-alive
```

★ Hands-on: simplest possible SSE example ★

4. Server (Express)

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();
app.use(cors());

app.get('/events', (req, res) =&gt; {
// 1. Set the SSE headers
res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');
res.setHeader('Connection', 'keep-alive');

// 2. Send messages every 2 seconds
let count = 0;
const interval = setInterval(() =&gt; {
res.write(`data: Message #${++count}\n\n`); // ← key line
}, 2000);

// 3. Cleanup on disconnect
req.on('close', () =&gt; {
clearInterval(interval);
res.end();
});
});

app.listen(3000, () =&gt; console.log('SSE server on :3000'));
```

■ Notice — no res.json() or res.send()

Just `res.write(...)` repeatedly. The connection stays open as long as you keep writing. The server doesn't "finish" the response until you call `res.end()`.

5. Client (React)

```
import { useEffect, useState } from 'react';

function App() {
const [messages, setMessages] = useState([]);

useEffect(() =&gt; {
const eventSource = new EventSource('http://localhost:3000/events');

eventSource.onmessage = (event) =&gt; {
setMessages(prev =&gt; [...prev, event.data]);
};

return () =&gt; eventSource.close(); // cleanup on unmount
}, []);

return (
&lt;ul&gt;
{messages.map((msg, i) =&gt; &lt;li key={i}&gt;{msg}&lt;/li&gt;)}
&lt;/ul&gt;
);
}
```

That's it. **5 lines** for the SSE part:

1. Create `new EventSource(url)`
2. Attach `onmessage` handler
3. Append data to state
4. Cleanup on unmount with `eventSource.close()`

No polling loops. No re-asking. The browser handles the connection, parses incoming chunks, and fires `onmessage` for each one.

6. What the browser actually does

```
1. EventSource opens GET /events (just an HTTP request)
2. Server replies with headers, but doesn't close
3. Server writes "data: foo\n\n" -> browser fires 'message' event with data='foo'
4. Server writes "data: bar\n\n" -> browser fires 'message' event with data='bar'
5. Server writes... -> browser fires...
6. (connection lost?) -> browser auto-reconnects after ~3 seconds!
```

> ■ **The killer feature**
>
> **Auto-reconnect is built in.** If the network drops, the browser silently retries the connection — no logic to write. WebSockets and long polling don't give you this for free.

7. Sending JSON

Most apps send JSON. Stringify on server, parse on client:

```
// Server
res.write(`data: ${JSON.stringify({ type: 'msg', user: 'Akshay', text: 'Hi!'
})}\n\n`);
```

```
// Client
eventSource.onmessage = (event) =&gt; {
const payload = JSON.parse(event.data);
console.log(payload.user, payload.text);
};
```

8. Named events (multiple channels in one stream)

Send different event types and listen for each separately:

```
// Server
res.write(`event: message\ndata: hello\n\n`);
res.write(`event: notification\ndata: ping!\n\n`);
```

```
// Client
eventSource.addEventListener('message', e =&gt; console.log('msg:', e.data));
eventSource.addEventListener('notification', e =&gt; console.log('notify:', e.data));
```

Useful for real apps — one stream, multiple kinds of updates.

9. Auto-reconnection with Last-Event-ID

If the connection drops, the browser automatically reconnects and sends the last seen ID in a header. The server can use it to resume from where it left off.

```
// Server
res.write(`id: 42\ndata: hello\n\n`); // include an id

// On reconnect, request comes with header:
// Last-Event-ID: 42
const lastId = req.headers['last-event-id'];
// resume sending events after id 42
```

This makes SSE robust for unreliable networks.

★ The real-time spectrum ★

10. Short Polling vs Long Polling vs SSE vs WebSockets

Aspect	Short Poll	Long Poll	SSE	WebSockets
Direction	C->S	C->S	**S->C only**	Both
Connections	Many short	One at a time	One long-lived	One long-lived
Protocol	HTTP	HTTP	HTTP	WS (upgraded HTTP)
Auto-reconnect	DIY	DIY	**Built-in**	DIY
Browser API	fetch	fetch	**EventSource**	WebSocket
Backend complexity	Easy	Medium	Easy	Medium
Through proxies?	Yes	Yes	Mostly yes	Sometimes no
Best use case	Status checks	Notif. fallback	**Live feeds, AI**	Chat, games

Decision rule

- Need **two-way** real-time? → WebSockets
- Need **server -> client** only? → **SSE** (simpler, auto-reconnects, plain HTTP)
- Updates rare? → Short polling (dead-simple)
- Stuck behind strict proxies? → Long polling fallback

11. Why SSE is great for AI chat streaming

When you see Claude or ChatGPT type out responses character-by-character, that's **almost certainly SSE**. The server streams tokens as they're generated by the LLM:

```
// OpenAI-style streaming with SSE
app.get('/chat', async (req, res) =&gt; {
res.setHeader('Content-Type', 'text/event-stream');
res.setHeader('Cache-Control', 'no-cache');

const stream = await openai.chat.completions.create({
model: 'gpt-4',
messages: [...],
stream: true,
});

for await (const chunk of stream) {
const token = chunk.choices[0]?.delta?.content || '';
res.write(`data: ${JSON.stringify({ token })}\n\n`);
}

res.write('data: [DONE]\n\n');
res.end();
});
```

> ■ **Familiar territory**
>
> You've used this pattern in MSG2AI — **Vercel AI SDK** and **OpenAI streaming** both use SSE under the hood. When you call streamText() and tokens appear in the UI, the underlying transport is SSE.

12. SSE limitations (must know)

1. **One-way only** — server -> client. To send anything back, use a normal fetch POST.
2. **6 connections per origin per browser** — HTTP/1.1 limit. Open more than 6 SSE streams to the same domain = blocked. With HTTP/2 the limit is dramatically higher.
3. **Text only** — no native binary support (use base64 if needed).
4. **No support in old IE** — but you're not targeting IE in 2026 anyway.
5. **Stateful connection** — same scaling considerations as long polling. Use a pub/sub layer (Redis, NATS) when you have multiple backend instances.

13. Common gotchas (interview-relevant)

Forgetting cleanup

```
// ■ Connection stays open after unmount → memory leak
useEffect(() =&gt; {
const es = new EventSource('/events');
es.onmessage = handler;
}, []);

// ■ Always close
useEffect(() =&gt; {
const es = new EventSource('/events');
es.onmessage = handler;
return () =&gt; es.close();
}, []);
```

Forgetting the double newline

```
res.write('data: hi'); // ■ no event fires
res.write('data: hi\n\n'); // ■ event fires
```

The double newline at the end of each event is **mandatory**. Browser won't fire `onmessage` without it.

Express buffering

Some setups buffer responses. To force flush:

```
res.flushHeaders(); // send headers immediately
res.write('data: ...\n\n');
```

Also be careful with `compression` middleware — it can buffer SSE traffic.

Authentication

`EventSource` doesn't support custom headers — including `Authorization`. Workarounds:

- Cookies (auto-sent)
- Token in query string (less secure)
- Use `fetch` with `ReadableStream` for full header control (modern alternative)

14. Modern alternative — fetch + ReadableStream

If you need custom headers (like Bearer tokens), you can do SSE-style streaming with `fetch`:

```
useEffect(() =&gt; {
const controller = new AbortController();

async function stream() {
const res = await fetch('/events', {
headers: { Authorization: `Bearer ${token}` },
signal: controller.signal,
});
const reader = res.body.getReader();
const decoder = new TextDecoder();

while (true) {
const { value, done } = await reader.read();
if (done) break;
const chunk = decoder.decode(value);
console.log(chunk);
}
}

stream();
return () =&gt; controller.abort();
}, []);
```

More flexible (custom headers, auth), but you write the parsing yourself — no auto-reconnect, no event types. **This is what the Vercel AI SDK uses internally** for AI streaming responses.

15. Interview Q&A

■ What is SSE?

Server-Sent Events — a web standard where the server pushes a stream of text events to the client over a single long-lived HTTP connection. The browser exposes it via the `EventSource` API. One-way only (server -> client), with built-in auto-reconnection.

■ SSE vs WebSockets?

SSE is HTTP-based, one-way (server -> client), simpler to implement, has built-in reconnection, and works well with existing infrastructure. WebSockets are bi-directional, slightly lower latency, and use a separate protocol after upgrading from HTTP. Use SSE when you only need server pushes; WebSockets when you need full duplex (chat, games).

■ How does SSE auto-reconnect?

When the connection drops, the browser automatically reconnects after a delay (default ~3s). It sends the last received event's ID in the `Last-Event-ID` header so the server can resume from where it left off.

■ Why do AI chat apps use SSE?

LLMs generate tokens incrementally, and users want to see text appearing character-by-character. SSE lets the server push each token as it's generated over a single connection. ChatGPT, Claude, OpenAI's API streaming, and Vercel AI SDK all use SSE internally.

■ Limitations of SSE?

One-way (server->client only), text-based (no binary), no custom headers in `EventSource` (so auth requires cookies or query strings), and limited to ~6 concurrent connections per origin in HTTP/1.1. For two-way streams use WebSockets; for full header control use fetch + ReadableStream.

■ How would you implement SSE for a multi-instance backend?

Use a pub/sub layer (Redis, NATS, Postgres LISTEN/NOTIFY). When an event happens on one backend instance, publish to the channel; all instances subscribe and forward to their connected SSE clients. Without this, a client connected to instance A won't receive events triggered on instance B.

■ How does the server "keep" the SSE connection open?

Same trick as long polling — it doesn't call `res.end()`. It sets the right headers, keeps writing chunks via `res.write(...)`, and Express keeps the TCP connection alive as long as the response isn't finalized. The handler may live for minutes or hours.

★ TL;DR ★

■ The whole story

SSE = server pushes a stream of text events to the client over a single long-lived HTTP connection.

- Client uses the `EventSource` API.
- Server writes `data: ...\n\n` repeatedly without closing the connection.
- Auto-reconnects on network failure.
- Works through proxies, plain HTTP.
- Perfect for live feeds, notifications, and **AI streaming**.
- Simpler than WebSockets when you only need server -> client.

One-line interview answer

■ If you only remember one sentence...

*"SSE is a standard for server-to-client streaming over a single long-lived HTTP connection. The server writes **data: ...\n\n** chunks; the client receives them via the EventSource API. It auto-reconnects, works over plain HTTP, and is what powers token-by-token streaming in AI chat apps."*

Reference for interview prep — Akshay Chavhan